PARKING LTD.

CAPSTONE MANUAL

INSTALLATION MANUAL

Local demonstration installation for the web application, SQLite database, computer vision, and VIGI NVR.

Automated Parking Management System

Group code: CAP-2520-IT

Prepared 9 November 2026 | Version 1.0

For developers and operators preparing the project on a macOS demonstration workstation.

1. Installation scope and prerequisites

This manual installs the repository as a local demonstration system. The application is plain HTML/CSS/JavaScript served by PHP, with a SQLite database and Python computer vision tools. No Node.js build is required. Live RTSP requires the workstation to reach the NVR network.

Component	Requirement / expected asset
Operating system	macOS workstation for the current VIGI and CV workflow; Linux may need platform-specific camera and PyTorch setup.
PHP	PHP 8.0 or newer with PDO SQLite enabled.
Python	Python 3.9+; project currently uses .venv-cv and was run under Python 3.12.
Frontend	Current browser, local PHP server, project frontend files.
Database	backend/database/parking.db and schema backend/database/Schema.sql.
Vision models	computer_vision/best.pt; yolov8n.pt; computer_vision/models/sectional_occupancy/weights/best.pt.
Vision packages	opencv-python, easyocr, ultralytics and compatible torch/torchvision dependencies.
Recordings	Chosen gate and sectional clips, plus calibrated region JSON for the selected zone.

> **Back up first**
>
> The project uses the supplied main SQLite database for the frontend and simulations. Back up backend/database/parking.db before a demonstration or schema migration. Runs, payments, alerts, and audit events may remain in this database.

2. Get the repository and verify files

Open Terminal at the repository root. Replace the example path if the repository is in a different location.

```
cd /Users/ridzrgt/Documents/GitHub/CAPSTONE/CAPSTONE
php -v
php -m | grep -Ei '^(PDO|pdo_sqlite|sqlite3)$'
ls -l backend/database/parking.db backend/database/Schema.sql
```

The PHP module output must include PDO and pdo_sqlite. The local server and API should run from the repository root so relative paths resolve. Do not create a new database for the standard demonstration flow.

3. Start the web application

```
php -S 127.0.0.1:8080 backend/router.php
```

For local reservation checkout, enable the simulator in the same terminal that starts PHP:

```
LOCAL_PAYMENTS_ENABLED=1 php -S 127.0.0.1:8080 backend/router.php
```

Open http://127.0.0.1:8080/ and sign in. Use the supplied development database or install
accounts with php backend/database/seed_demo_accounts.php. Do not use publicly visible de
on a production host. Keep this server local to a trusted demo network.

4. Prepare the Python environment

```
python3 -m venv .venv-cv
source .venv-cv/bin/activate
python -m pip install --upgrade pip
python -m pip install opencv-python easyocr ultralytics
```

Install a PyTorch build compatible with the workstation and the installed Ultralytics ver
resolver did not install one. Then verify imports and model paths:

```
python -c "import cv2, easyocr, torch, ultralytics; print(cv2.__version__,
torch.__version__, ultralytics.__version__)"
ls -l computer_vision/best.pt yolov8n.pt
computer_vision/models/sectional_occupancy/weights/best.pt backend/database/parking.db
```

Model roles

The sectional checkpoint locates bay regions. yolov8n.pt detects vehicles and supports color
best.pt is used for gate plate detection/OCR. Model accuracy depends on camera view, focus, l
quality, and calibration.

5. Create the camera event token

The camera-facing API requires a shared secret of at least 32 characters. Create the loca
file is missing:

```
php backend/database/create_exit_camera_token.php
```

The API and CV code read backend/database/exit_camera_token by default. Keep it private a
version control or screenshots. If PARKING_EXIT_EVENT_TOKEN is used instead, PHP and Pyth
receive the exact same value. Do not paste the token into a document, source file, termin
commit.

6. Configure and reach the VIGI NVR

Setting	Current demonstration value
NVR address	192.168.31.2
NVR username	admin

Setting	Current demonstration value
RTSP port	554 unless configured otherwise
Entrance feed	Channel 1, primary stream
Exit feed	Channel 13, primary stream
RTSP path	/live/<channel>/1/avm, constructed by live_vigi.py

```
ipconfig getifaddr en0
nc -G 3 -vz 192.168.31.2 554
source .venv-cv/bin/activate
export VIGI_NVR_IP="192.168.31.2"
export VIGI_NVR_USERNAME="admin"
read -s 'VIGI_NVR_PASSWORD?NVR password: '
export VIGI_NVR_PASSWORD
echo
```

A successful TCP connection is required before RTSP will work. If the workstation is on a different subnet, configure network access or routing first. Never put the password in the RTSP URL in a shell command or commit it to the repository. The NVR account must be permitted to view both channels over RTSP.

7. Validate the feeds before recording events

```
python computer_vision/live_vigi.py --detection-only
```

Expect channel 1 and 13 connection messages and preview windows. Detection-only mode publishes no entry/exit events. Press q in the preview or Ctrl+C to stop. If either stream times out, confirm IP/subnet, TCP port, account permission, NVR firmware path, and channel availability. If video connects but no plate confirms, inspect the view and exposure before changing thresholds.

8. Run direct live gates

```
python computer_vision/live_vigi.py
```

A confirmed entrance detection posts a channel 1 event to the local PHP API. The API selects an eligible vacant bay using database occupancy/reservations, creates a PARKED session, records the plate/color and timestamps, and returns an entry QR. A confirmed channel 13 exit requests clearance and closes an eligible session. The guard still operates the barrier.

9. Run recorded and combined simulation modes

Activate .venv-cv and ensure PHP/API is available. The runner uses the main database and starts a local PHP server if needed. For an ordinary recorded run:

```
python computer_vision/recorded_test_cases/grace.py --zone A
```

The six per-rule entry points are grace.py, normal_fare.py, reservation.py, reserved_slot.py, tenant.py, and enforcement.py (Zone C clip only). Business-rule tests operate Zone A through E; recorded roadway enforcement footage currently exists only for Zone C. A complete example appears in the recorded run guide in the repository.

To use live gates with a recorded sectional clip, invoke full_simulation.py live-gates and provide the clip, bay map, and test identity policy required by that scenario. Do not assign a clip plate label to an unrelated vehicle. See docs/LIVE_VIGI_SETUP.md and docs/SECTIONAL_FIRST_SIMULATION.md for current command examples.

10. Stop, recover, and preserve data

- Stop PHP with Ctrl+C in its terminal and stop live vision with q or Ctrl+C.
- If a simulation ends before exit, resolve payment and pending staff review. An administrator can use Simulation Sessions recovery with a reason; it is an administrative recovery, not an observed camera departure.
- Back up the database with the application stopped or using SQLite backup tooling. Keep the database file and camera token private. Evidence files are stored under backend/database/simulation_runs/<scenario-id>/.
- Do not copy the token or a database containing personal/vehicle records into public repositories or shared demo materials unless approved and sanitized.

11. Installation troubleshooting

Issue	Resolution
PHP API says database connection failed	Check pdo_sqlite, database file permissions, working directory, and PARKING_DB_PATH if explicitly set.
Port 8080 busy	Stop the existing PHP process or choose another port and update the browser/API URLs consistently.
OpenCV import or torch errors	Rebuild .venv-cv with compatible Python, PyTorch, and Ultralytics versions; do not mix system and virtualenv packages.
Missing model file	Restore the project model asset to the documented path. Check model storage or Git LFS status if applicable.
NVR connect timeout	Check subnet/routing and TCP port 554 before changing model or OCR settings.
Camera API authorization failed	Confirm token file exists, is at least 32 characters, and matches the PHP environment token if overridden.
No preview on a headless workstation	Use --no-preview where supported; use a desktop session for key controls u and r.